A. Proposed Approach

Our project tackles the challenge of building a secure, multi-tenant learning platform where users can have different roles across various organizations. To address this, we decided to use a modular monolith architecture. This approach is better than managing multiple microservices because it reduces the complexity for our small team while still keeping the code organized in logical modules.

For identity management, we chose to integrate Keycloak (version 26.8.0). Building a custom identity provider is risky and takes too much time. Keycloak handles OIDC login, token issuance, and session lifecycles out of the box. For authorization, we evaluated Keycloak's built-in policies but found them awkward for our tenant-specific and relationship-based rules (like "an instructor can only grade courses they teach"). Therefore, we integrated Casbin as our declarative authorization engine. This allows us to define clear, testable role-based policies while keeping the tenant and relationship facts in our application database (PostgreSQL).

B. System Design

The system consists of several interacting components deployed via Docker Compose:

- **Clients:** We have lightweight web and mobile-style browser clients that use Authorization Code Flow with PKCE to authenticate securely.
- **Identity Provider (Keycloak):** Manages user credentials, issues JWT access tokens, and handles enterprise federation.
- **Backend API (Fastify):** A Node.js server that validates incoming tokens using Keycloak's JWKS. It uses custom middleware to map the token's subject to our internal user ID and verifies the requested tenant context.
- **Database (PostgreSQL):** Stores application state including users, organizations, memberships, learning resources, submissions, consent, and queryable audit logs. It enforces data integrity using unique constraints and composite foreign keys.

Workflow: A user logs in via the client and gets an access token from Keycloak. The client sends this token and an `X-Organization-ID` header to the Fastify API. The backend verifies the token, checks if the user has an active membership in that organization, and then Casbin evaluates if the user's role permits the requested action. Finally, the API performs the action and logs an audit event.

C. Technical Feasibility

Tools and Frameworks:

- Node.js (v24), Fastify (v5.12.5) for the backend.
- PostgreSQL (v16.15) with Kysely for type-safe database interactions.
- Keycloak (v26.8.0) for identity management.

- Casbin (v5.51.1) for authorization.
- Docker and Docker Compose for reproducible local infrastructure.
- Vitest and Playwright for automated security and browser testing.

Feasibility and Challenges: The design is technically feasible as it relies on mature, well-supported open-source tools. One challenge was handling cross-tenant data leaks. We mitigated this by ensuring every database query is strictly filtered by the selected organization ID. Another challenge was ensuring that token revocation works properly. We found that a password reset does not automatically revoke an existing access session, so incident response must explicitly trigger a logout-all.

D. Implementation Plan

Completed Work:

- Infrastructure setup with Docker Compose (PostgreSQL and Keycloak).
- Database migrations with composite constraints and initial seed data.
- Fastify backend with token validation, tenant context middleware, and audit logging.
- Casbin integration for declarative RBAC policies.
- API endpoints for learning resources, submissions, and member management.
- Automated security test suite (159 tests passing, covering authorization, tenant isolation, and session lifecycle).
- Real-browser PKCE login verification.

Ongoing and Pending Tasks:

- *Ongoing:* Refining the documentation and demo guides.
- *Pending:* Implementation of optional extensions such as application machine identities (service principals), user-delegated agents, and full SCIM provisioning.
- *Pending:* Further hardening for production deployment, such as secret management and TLS configuration (currently everything runs locally for the academic prototype).

Expected Outcomes: The expected outcome is a fully functional, secure prototype demonstrating proper tenant isolation, robust authorization, and clear audit trails, ready for final evaluation and demonstration to our mentors.